

Learning Resource On Speech and Video Processing

Module-2: Feature Extraction and Speech Recognition

Chapter-7: Pattern Matching and HMM based Speech Recognition

By:

Asst. Professor

SCE, KIIT, DU, Bhubaneswar-24

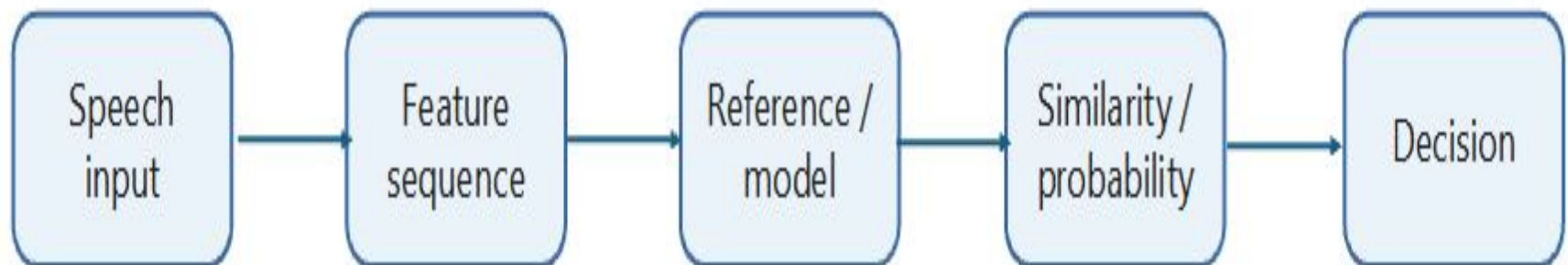
Chapter Outcomes

- After completing this module, students will be able to:
 - Explain the significance of pattern matching in speech recognition.
 - Describe different pattern matching techniques used for recognizing speech patterns.
 - Explain the basic concepts and components of Hidden Markov Models (HMMs).
 - Describe how HMMs are used to model and recognize speech sequences.
 - Apply basic HMM-based speech recognition concepts to simple speech recognition problems.

Pattern Matching

- **Pattern matching** is a fundamental process in speech recognition that helps **identify** an **unknown speech input** by comparing its **acoustic feature patterns** with stored **reference patterns or trained models**.
- Since speech is **highly variable** due to differences in speakers, speaking rate, pronunciation, noise, and linguistic context, pattern matching provides a systematic way to handle these variations and determine the most **likely word or phoneme**.

- In **speech recognition**, the input is typically represented as a sequence of **acoustic feature vectors**, and the system selects the **word or phoneme** whose model provides the **best overall match**.
- Thus, effective pattern matching is essential for achieving **accurate and reliable** speech recognition.



Pattern Matching Approaches

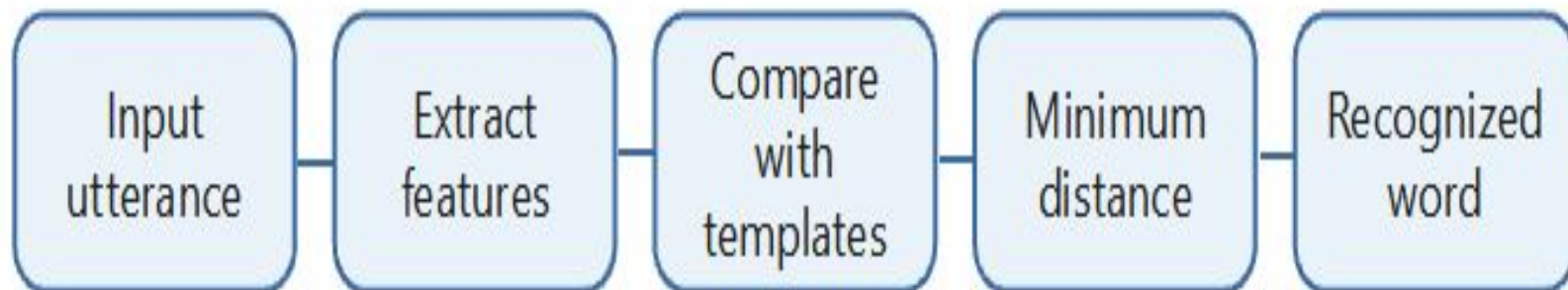
Pattern Recognition Approaches can be categorized as

- **Template-based Pattern Matching**
 - Distance-based, Dynamic Time Warping(DTW)
- **Statistical/ Probabilistic Modeling**
 - Hidden Markov Model (HMM), Gaussian Mixture Model (GMM)
- **Model-based recognition/ knowledge-based**
 - Phonetic/linguistic models, knowledge-based models

Pattern Matching Techniques

- **Template Matching**

- A reference template is stored for each word or pattern.
- The unknown utterance is compared directly with each template.
- The smallest distance (or largest similarity) determines the winner.
- Simple but sensitive to speaker and speaking-rate variations

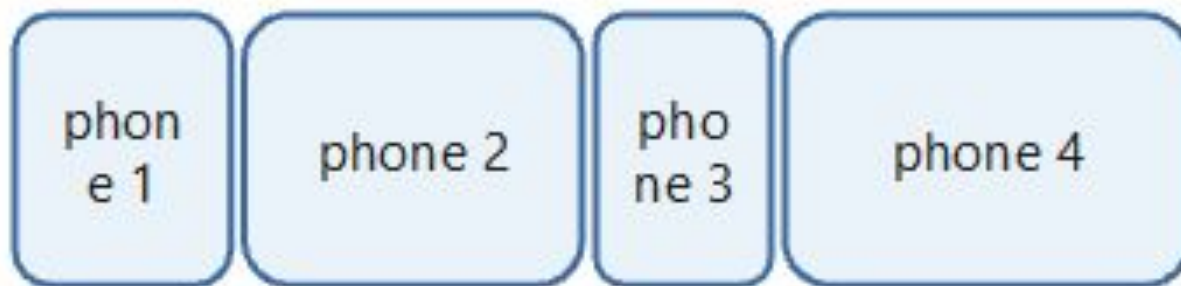


Distance Measures

- A key component of speech pattern matching is measuring the **dissimilarity** between two feature vectors.
- The **distance measure** should ideally reflect the perceptual or subjective differences between speech signals.
- Traditional measures such as **Euclidean distance** or **squared error** may not adequately capture the perceptual importance of different speech features.
- Therefore, specialized **spectral distortion measures** as mentioned below, are used in speech processing
 - Log Spectral Distance; Cepstral Distance; Weighted Cepstral Distance; Likelihood Ratio Distortion; Weighted Likelihood Ratio Distortion, and Weighted Slope Metric.

Why Simply Matching Fails for Speech?

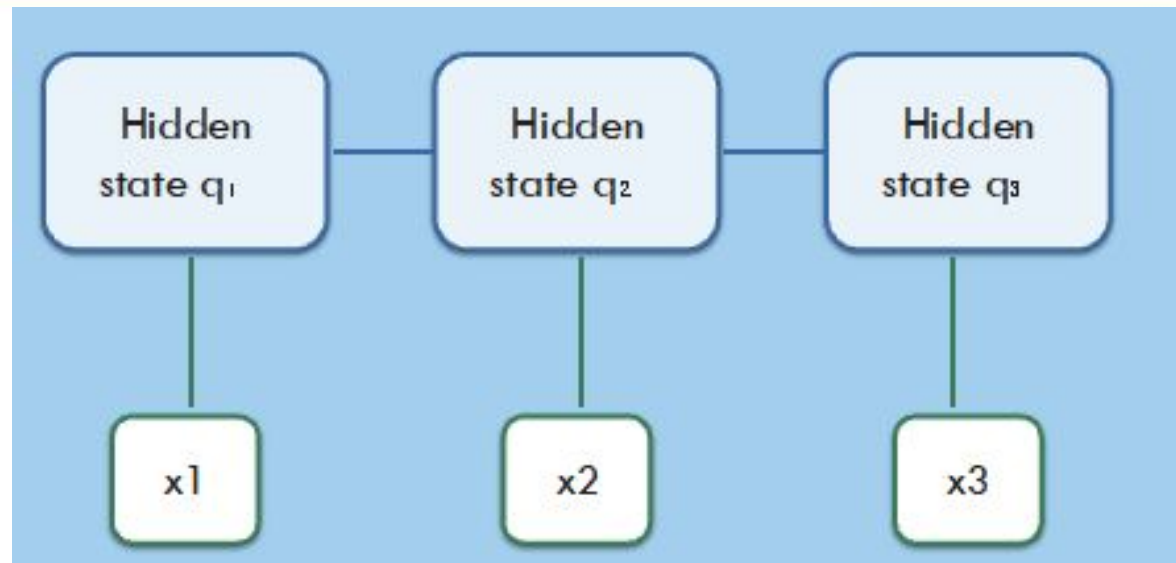
- Two people can pronounce the same word with different acoustic characteristics.
- The same speaker can say a word at different speeds.
- Some phonemes occupy more frames than others.
- Noise and channel conditions alter feature vectors.
- Therefore, temporal alignment and probabilistic modeling are needed.



Different durations → alignment problem

Hidden Markov Model

- A **Hidden Markov Model (HMM)** represents speech as a sequence of hidden states and observable acoustic feature vectors.
- The **hidden states** represent different **temporal parts** of a speech unit, such as the **beginning, middle, and end of a phoneme**.
- The model describes how speech moves from one state to another using transition probabilities.
- It also uses **emission probabilities** to represent how likely a particular acoustic feature is to occur in each state.
- The states are called hidden because they **cannot be directly observed**; only the corresponding acoustic features are observed.



- **Hidden States (q_1, q_2, q_3):** These represent the underlying process that **cannot be directly observed**, with the **connections** between states representing **transition probabilities** from one state to the next.
- **Observations (x_1, x_2, x_3):** These are the **visible data** generated at each time step, with the **connections** from hidden states to observations representing **emission probabilities**

Parameters of HMM

An HMM is represented by $\lambda = (A, B, \pi)$ and consists of following parameters:

- **Observation sequence:** $X = \{x_1, x_2, \dots, x_t, \dots, x_T\}$; where x_T is the observed acoustic feature at time (t).
- **States** $S = \{S_1, \dots, S_N\}$: representing the hidden temporal conditions of the speech signal.
- **Initial state probability (π)** $\pi_i = P(q_1 = S_i)$; the probability that the model starts in state S_i
- **Transition probability (A)** $a_{ij} = P(q_{t+1} = S_j | q_t = S_i)$; the probability of moving from state S_i to S_j
- **Observation Emission probability (B)**
$$b_j(x_t) = P(x_t | q_t = S_j)$$

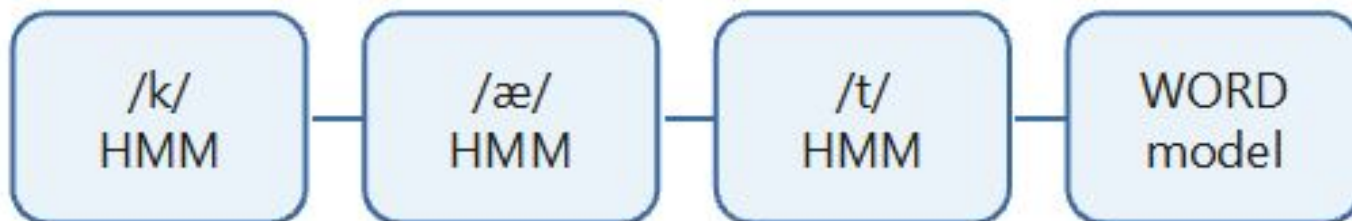
the probability of observing x_t when the system is in state S_j .

- The **Hidden Markov Model (HMM)** has three main problems:
 - **Evaluation ($P(X|\lambda)$):** Given an **observation sequence** and a **trained model**, what is the **probability** that the model generated this **sequence**?
 - **Algorithm:** Forward Algorithm.
 - **Decoding state path:** Given an **observation sequence** and a **model**, what is the **single most likely sequence** of hidden states that produced those **observations**?
 - **Algorithm:** Viterbi Algorithm.
 - **Learning Estimate λ :** Given a set of **observation sequences**, how to estimate the **model's parameters** (transition probabilities A , emission probabilities B , and initial state distribution π) to **best fit the data**.

Left-to-Right HMM for Speech

- Speech recognition typically relies on **left-to-right (Bakis) HMMs** to capture how speech unfolds over time.
- **States** progress naturally from the **beginning** to the **middle and end** of a speech unit, with backward transitions generally disallowed in basic configurations.
- A standard **phoneme model** utilizes three emitting states representing the **onset, steady state, and offset** of an acoustic region.
- **Word-level models** are constructed by concatenating individual phoneme HMMs.
- A **word HMM** is a composite HMM constructed to represent an entire spoken word by chaining together smaller sub-word units, typically phonemes.

- **Phoneme Models:** Each box represents a HMM trained for a specific phoneme (individual sound).
 - **The Sequence:** The phonetic transcription represents the English word "cat":
 - /k/ — the initial consonant sound
 - /æ/ — the short vowel sound
 - /t/ — the final consonant sound
- **Word Model:** Connecting these individual phone HMMs in a linear sequence forms the complete model used to recognize that specific word.



Forward Algorithm

- A **dynamic programming** method used in HMMs to compute the **total probability** of generating a specific observation sequence given a model ($P(\mathbf{O}|\lambda)$).
- It uses a **forward variable**, $\alpha_t(i)$, which represents the joint probability of the partial observation sequence up to time t and being in hidden state S_i at time t .

- **Initialization** ($t = 1$): $\alpha_1(i) = \pi_i b_i(o_1)$

- **Recursion** ($t = 2$ to T):

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t)$$

- **Termination:**

$$P(\mathbf{O}|\lambda) = \sum_{i=1}^N \alpha_T(i)$$

- **Initialization (The Starting Point):** At the very first moment ($t = 1$), calculate the starting score for each state.
 - To do this, multiply the probability that state i produces for first observation ($b_i \mathbf{O}_1$) with the probability of starting in state i i.e., (π_i)
- **Recursion (Moving Forward):** For every step from time 2 until the end (T), calculate the new scores based on the old ones.
 - **Sum (Σ):** Look at all states (i) you could have just come from.
 - $\alpha_{t-1}(i) \cdot a_{ij}$: Multiply their previous scores by the probability of transitioning from state (i) to your new state (j).
 - $b_j(\mathbf{O}_t)$: Multiply that sum by the probability that your

- **Termination (The Final Total):** Once you reach the final time step (T), you have a list of final scores for every possible ending state.
 - **Action:** Add all of those final scores together.
 - **Result:** This grand total is the complete probability ($P(O|\lambda)$) of that entire sequence happening.
- **Sample Example:** Given a two-state HMM with hidden states S_1 and S_2 , an initial state probability vector $\pi = [0.6, 0.4]$, and a state transition probability matrix $A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$, calculate the total probability $P(X|\lambda)$ of generating the specific sequence of two observations $X = (x_1, x_2)$ using the Forward Algorithm, where the emission probabilities for the first observation are $b_1(x_1) = 0.5$ and $b_2(x_1) = 0.4$ and for the second observation are $b_1(x_2) = 0.6$ and $b_2(x_2) = 0.3$.

- **Step 1: Initialization ($t = 1$), calculate**
 - the starting probability score (α_1) for each state using the first observation x_1 .
 - **State 1:** $\alpha_1(1) = \text{Initial } \pi_1 \times \text{Emission } b_1(x_1) = 0.6 \times 0.5 = 0.30$
 - **State 2:** $\alpha_1(2) = \text{Initial } \pi_2 \times \text{Emission } b_2(x_1) = 0.4 \times 0.1 = 0.04$
- **Step 2: Recursion ($t = 2$)** calculate the scores for the next time step (α_2) by looking at all paths coming from $t = 1$ and multiplying by the new emission rates for x_2 .
 - **For State 1: $\alpha_2(1)$**
 - From State 1: $0.30 \times 0.7 = 0.21$
 - From State 2: $0.04 \times 0.4 = 0.016$
 - Sum them up and multiply by emission $b_1(x_2)$.

- • **For State 2: $\alpha_2(2)$**
 - From State 1: $0.30 \times 0.3 = 0.09$
 - From State 2: $0.04 \times 0.6 = 0.024$
 - Sum them up and multiply by emission $b_2(x_2)$:
 $(0.09 + 0.024) \times 0.5 = 0.057$
- **Step 3: Termination (Final Answer):** To get the total probability of generating the exact sequence x_1, x_2 , we add the final scores of all possible ending states together:
 - $P(\mathbf{X}|\lambda) = 0.0904 + 0.057 = \mathbf{0.1474}$

Viterbi Algorithm

- Viterbi finds the **single most probable sequence** of hidden states that can generate a given sequence of observations.
- The **single optimal path** (the maximum likelihood path) is also known as **Viterbi path**.
- It uses **dynamic programming** to find the Viterbi path by keeping track of the **best score at each step** and **discarding** sub-optimal paths at every step.
 - When a user speaks a sentence (utterance), the system receives a sequence of **acoustic feature frames (observations)**.
 - The system uses **concatenated phoneme** and **word HMMs** to represent possible spoken words, creating a massive network of **hidden states**.
 - The Viterbi algorithm efficiently navigates this massive network to extract the **single most probable string** of words or phonemes, handling temporal alignment.

- **Initialization ($t = 1$):** Computes the starting probability for each hidden state (i).
 $\delta_1(i) = \pi_i X b_i(o_1)$; where, π_i is the initial state probability; $b_i(o_1)$ is the emission probability of first observation.
- **Recursion ($t=2$ to T):** Calculates the highest probability $\delta_t(j)$ of reaching state (j) at time step (t).
 - Scans all possible previous states (i) to find the path that yields the highest transition probability (a_{ij})
$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] b_j(o_t)$$
 - **Backtracking Storage:** Stores the identity of that best previous state in a tracking variable $\psi_t(j)$, so the path can be reconstructed later.

- **Termination:** At the final frame,
 - select the largest $\delta_{T(j)}$, then
 - backtrack using ψ from T to 1 to reconstruct the optimal state sequence.
- **Sample Example:** Given a two-state HMM with hidden states S_1 and S_2 , an initial state probability vector $\boldsymbol{\pi} = [0.6, 0.4]$, and a state transition probability matrix $\mathbf{A} = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$, calculate the total probability $P(\mathbf{X}|\boldsymbol{\lambda})$ of generating the specific sequence of two observations $\mathbf{X} = (x_1, x_2)$ using the **Viterbi Algorithm**, where the emission probabilities for the first observation are $b_1(x_1) = 0.5$ and $b_2(x_1) = 0.1$ and for the second observation are

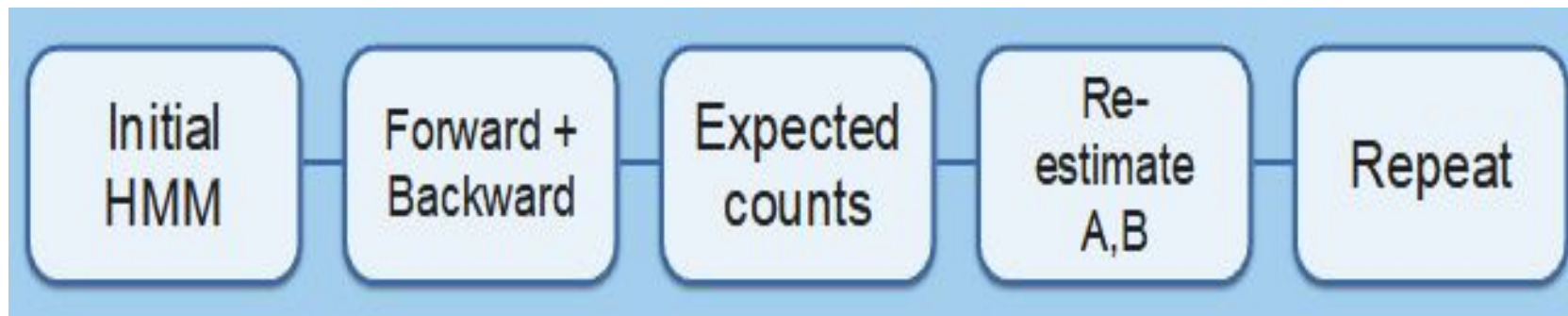
- **Step 1: Initialization ($t = 1$),** calculate the starting probability score (α_1) for each state using the first observation x_1 .
 - **State 1:** $\alpha_1(1) = \text{Initial } \pi_1 \times \text{Emission } b_1(x_1) = 0.6 \times 0.5 = 0.30$
 - **State 2:** $\alpha_1(2) = \text{Initial } \pi_2 \times \text{Emission } b_2(x_1) = 0.4 \times 0.1 = 0.04$
- **Step 2: Recursion ($t = 2$):** Viterbi Algorithm uses the max function to keep only the single best path leading into each state.
 - $t = 2$, **state 1:** $\delta_2(1) = \max(0.30 \times 0.7, 0.04 \times 0.4) \times 0.4 = \max(0.21, 0.016) \times 0.4 = \mathbf{0.084}$
 - $t = 2$, **state 2:** $\delta_2(2) = \max(0.30 \times 0.3, 0.04 \times 0.6) \times$

• Step 3: Termination & Backtracking

- **Find the Best Ending State:** Looking at the final scores at $(t=2)$, we get $\max(0.084, 0.045) = 0.084$, which points directly to S_1 .
- **Trace Back the Path:** Looking back at who the predecessor was for S_1 at step 2, it was S_1 .
- The most likely sequence of hidden states that produced the observations (x_1, x_2) is: $S_1 \rightarrow S_1$
- **Note:** While the Forward Algorithm calculates the total combined probability of an entire observation sequence across all possible paths, the Viterbi Algorithm isolates only the single most likely sequence of hidden states that

Baum_Welch Algorithm

- **Training** estimates HMM parameters from acoustic training sequences.
- **Forward-backward** computes state occupancy probabilities.
- Parameters are **re-estimated iteratively** to increase **training-data likelihood**.
- This is an instance of **Expectation-Maximization (EM)**.
- In classical ASR, many HMM states use Gaussian or GMM emission models.



Continuous and Discrete HMM

- For a **discrete HMM**

- If the observations are **discrete**, $b_{qt}(x_t)$ is a **probability mass function**.
- Discrete HMMs require **vector quantization** before modeling.

- For a **continuous HMM**

- If observations are **continuous** feature vectors, $b_{qt}(x_t)$ is generally a **probability density**, often modelled using a **Gaussian or Gaussian mixture**:
$$b_j(x_t) = N(x_t; \mu_j, \Sigma_j).$$
- GMM-HMMs provide flexible multimodal emission distributions.